

Experiment 9 — First and second HPC sweeps: 43 classical kernels

Mathilda — execution-speed experiments

Contents

Experiment 9 — First and second HPC sweeps: 43 classical kernels	1
Hypothesis	1
What was built	1
What it found	2
Results	2
What the second sweep added	3
Still open	3
Where the slow rows went	3

Experiment 9 — First and second HPC sweeps: 43 classical kernels

Date: 2026-07-30 → 2026-07-31 · Commits: 6c78653 and follow-ups · Harness: [comparisons/hpc_bench.py](#)
· Report: [docs/design/performance.md](#) §2–§8
Common method in [README.md](#).

Hypothesis

Experiments 1–8 each fixed something a profile pointed at. A profile only shows you the code you are already running, which means it cannot find the thing you are doing 40× too slowly everywhere, because that cost is spread evenly and looks like the baseline.

The hypothesis was that a broad, adversarial benchmark against a mature competitor would find a different class of problem than profiling does — and specifically that it would find fast paths that exist and are never reached.

What was built

38 kernels in the first pass, 5 more in the second, across:

dense linear algebra · spectral transforms · STREAM-style array traffic · PDE stencils · compiled scalar loops · integer and combinatorial search · arbitrary-precision arithmetic · a Krylov solver · direct convolution · ODE time integration · an irregular hash-keyed reduction · interpolation

Plus the harness discipline that made the numbers trustworthy — all of it retained for the later sweeps:

- **AbsoluteTiming**, never **Timing[]**. **Timing[A . A]** on a 600×600 reads 10.5–13.7 ms against a true 1.9 ms, because Accelerate's dgemm is threaded and **Timing** sums CPU over threads. Any comparison of a threaded path against a serial one using **Timing[]** is wrong by roughly the core count.
- **AbsoluteTiming** did not exist in Mathilda and was written for this.

- Min of N reps, max recorded as a tripwire. A builtin with an internal cache answers the second rep in ~0 s; reporting the min alone makes the row vacuous.
- Values must be forced. Wolfram evaluates $(p + k) q$; on two 10^6 -bit integers in 138 μ s and `IntegerLength[(p + k) q]` in 2.96 ms — the first never materialises the product. Every row was re-checked in both forms.
- Checked answers. Each kernel's scalar result is compared across systems.

What it found

The 40× and 80× rows: a complete fast path nobody could reach

Fourier was the worst result in the document. `fourier.c` had had a full NDArr implementation all along — and the packing transparency gate materialised the buffer into 2^{20} Expr nodes before the builtin could see it.

`Fourier[packedList]` — 986 ms. `Fourier[NDArr[...]]` — 62.8 ms. On identical data.

Nothing was slow. The fast path was simply unreachable, because the head was not on the aware list. Two more causes on the same row (an identity b-gather doing an integer division and a modulo per element per axis, and a bounce through a second buffer) took it to within 1.15×/1.30× of Mathematica.

This is the finding the experiment was for, and it recurred: the same shape hit `Nest`, then `Outer`, then `ListConvolve`. `make check-packed-aware` exists because of it.

Interpolation was quadratic in its own table

151 μ s per point at 10^4 nodes, 906 μ s at 5×10^4 . At 10^5 nodes the benchmark could not be timed at all — 228 s.

Four per-point costs, none depending on the point. Three were table-wide facts recomputed per call, now memoised on the table's identity (sound because `expr_copy` is a refcount bump, so the memo holds a reference). The fourth was a genuine $O(n^2)$: `build_grid` inserted coordinates one at a time into a sorted array scanning from the front, so coordinates arriving in order — which is every table from `Table[{x, f[x]}, {x, a, b, dx}]` — walked the whole grid on every insertion. Sort-and-unique instead: the 160000-node grid build went from 4.96 s to 21 ms.

1.494 s $\rightarrow$ 7.65 ms (195×), and from 74× slower than Wolfram to 2.6× faster.

Tally was slower packed than unpacked

No algorithm to fix — `Tally` already hashes. The cost was boxing: it reads `list->args[i]`, so a packed buffer was materialised into one Expr per element first, and every probe hashed and compared Expr nodes. Over 10^6 packed int64: 147 ms packed against 96 ms for the identical data as a plain **List**.

A representation that is a net loss is the failure mode worth having a benchmark for. It is invisible to profiling, because the profile of the packed run looks exactly like the profile of the unpacked one, only longer.

The **PadRight** row is a semantic fact, not an inefficiency

`PadRight[{1., 2., 3.}, 5]` is `{1., 2., 3., 0, 0}` — exact Integer zeros beside Reals — in both systems, so neither can hold it in a uniform buffer and both fall back to a boxed list. Given an explicit Real fill, Mathilda is the faster of the two by 1.29×.

Worth stating because it is the kind of row that invites a "fix" that would change an answer.

Results

Full tables in [performance.md](#) §2–§8. The shape:

Benchmark	Mathilda	Mathematica 14.0	NumPy / Python	vs WL	vs Nu
Matrix multiply, 1000x1000	7.42 ms	6.46 ms	7.07 ms	1/1.15x	1/1.15x
Fourier, 2 ²⁰ reals	22.58 ms	20.99 ms	40.75 ms	1/1.08x	1/1.08x
Sort	318.55 ms	714.17 ms	155.76 ms	2.24x	1/2.24x
Jacobi 5-point relaxation, 512 ² , 100 sweeps	223.01 ms	110.71 ms	68.78 ms	1/2.01x	1/3.01x
Mandelbrot, 800x800, 100 iterations	769.23 ms	1.621 s	980.20 ms	2.11x	1/2.11x
Sieve of Eratosthenes to 10 ⁷	664.56 ms	784.49 ms	26.44 ms	1.18x	1/25.18x
50000! (exact)	2.62 ms	2.29 ms	45.16 ms	1/1.14x	1/1.14x
Conjugate gradient, 2D Poisson 256 ² , 100 iterations	147.90 ms	82.44 ms	45.09 ms	1/1.79x	1/3.79x
ListConvolve, 1024 ² image, 5x5 kernel	35.67 ms	33.82 ms	39.71 ms	1/1.05x	1/1.05x
NDSolve, Lorenz system to t = 200	26.70 ms	47.78 ms	962.09 ms	1.79x	36.47x
Interpolation, 10 ⁴ nodes, 10 ⁴ evaluations	6.39 ms	20.00 ms	798 us	3.13x	1/8.13x
Tally, 10 ⁷ integers into 10 ⁴ bins	122.08 ms	17.03 ms	106.48 ms	1/7.17x	1/7.17x

Caveat on the Jacobi row. It reads 223 ms above and 128 ms measured alone. Bisecting the benchmark prefix found the trigger: running one 1000×1000 dgemm first makes Mathilda's threaded stencil 1.45× slower, reproducibly. See [HPC_SWEEP_3_NUMPY_GAP.md](#) §"A separate finding". Both threaded-stencil rows (Jacobi, Game of Life) follow the linear-algebra group in the run order and should be read with that in mind.

What the second sweep added

Five kernels chosen to run where none of the 38 did. Two came back faster than Wolfram with no work at all — NDSolve at 1.52×, which is the RHS compiler paying off on the workload it was built for.

The important one was conjugate gradient, which was slower than Wolfram despite every one of its parts being at or ahead of parity. A composition can be slow in ways none of its components are. That single observation is why the third sweep is built out of application pipelines rather than primitives — see [HPC_SWEEP_2_APPLICATIONS.md](#).

Still open

The gate allowlist in `hpc_bench.py` is a to-do list, not a settled state. Remaining from these two sweeps: ListConvolve (addressed in experiment 11), Tally's hash probe, and the Reverse/RotateLeft/Differences/Accumulate band of serial buffer work — partly addressed in experiment 11, partly still wanting SIMD.

Where the slow rows went

This sweep's job was to find the gaps, not to close them; every row it identified is carried by a later experiment, and the roadmap for each lives there rather than being duplicated here:

what this sweep found	now tracked in
QRDecomposition, Eigenvalues, SVD bypass LAPACK (6–14×)	07-blas-lapack-routing/
The sieve at 25× NumPy, Game of Life at 2.8×, fib at 30×	05-machine-integers/
Reverse/RotateLeft serial, Sort at 2×	06-packed-arrays/
Sin/Exp at 1.16× Mathematica	02-compile-array-fusion/
Structural ops and scans on the boxed path	11-hpc-sweep-numpy-gap/
Compositions slower than their parts	10-hpc-sweep-applications/

The one thing this sweep should be read for on its own is the method: a wide net over seven categories, run against two other systems, with a value check on every row. Five of the six entries above were invisible until something outside the tuned path was measured.